

Benjamin Tran-Pugh

btranpug@uwaterloo.ca | [linkedin.com/in/benjamin-tran-pugh](https://www.linkedin.com/in/benjamin-tran-pugh) | github.com/BenjaminTP | benjamintranpugh.com

PROJECT PORTFOLIO

Active Suspension: Phase 2

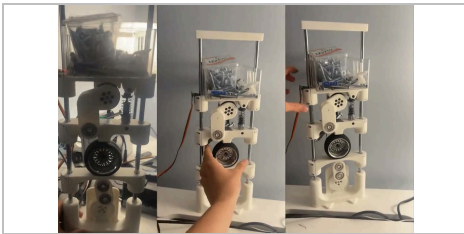
SolidWorks, 3D Printing, Direct-Drive Motor, IMU, Hardware

BUILT IT FOR REAL

AUG 2026

Building the phase 1 model into a physical rig. Friction and motor cogging overdamped the system, so skyhook could not run, but the build taught me how to design around imperfect hardware.

- **Goal:** Phase 1 proved skyhook control in simulation. Phase 2 was meant to build that quarter-car model in hardware and run the controller for real. I built the rig. The controller never ran: friction and motor cogging overdamped the system, and skyhook is undefined on an already-overdamped plant. This page is the honest version of that, because the build is where I learned the most.
- **Method:** I needed every element of the phase 1 model as a physical part, constrained to move only vertically on linear rails. The piece with no obvious answer was the road: I turned a servo through a crank and pushrod to convert rotation into vertical travel, and used the same trick for the skyhook actuator.
- **Outcome:** I did not get skyhook running on hardware, and I am not going to pretend otherwise. What I got was the part that does not show up in a simulation: where the model quietly assumes an ideal, and how to design around hardware that refuses to be ideal. The full parts list, measurements, and derivations are in the Phase 2 writeup.



THE RIG RUNNING / 3 HZ ROAD, DROP TEST, PUSH DOWN AND PULL UP



FINAL RIG / MODELLED IN SOLIDWORKS



THE ASSEMBLED RIG

Full write-up and photos: benjamintranpugh.com/projects/active-suspension-phase-2/

Active Suspension: Phase 1

Python, Control Systems, Vehicle Dynamics, ODEs, ISO 8608

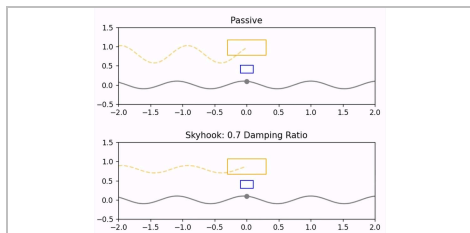
RMS ACCEL: -9.6% TO -57.1% | ACTUATOR: 663 N PEAK

JUL 2026

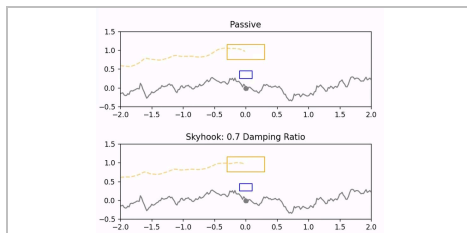
Skyhook control on a quarter-car model, cutting RMS body acceleration by 9.6% on a worn road and 57.1% at resonance.

- **Goal:** I built a quarter-car suspension model from scratch to answer one question: how much ride comfort can an actuator buy you over a passive spring and damper? Comfort is measured as RMS body acceleration, so that is the number I optimized and the number I report.
- **Method:** Most derivations I found waved away gravity, so I worked the equations of motion out from first principles instead. Writing the spring forces in absolute coordinates, then substituting each mass's static equilibrium position, makes the mg terms cancel exactly. That is the reason gravity drops out, and it also fixes what the coordinates mean: every displacement below is measured from static equilibrium, not from the ground.
- **Outcome:** This page is the summary. I wrote a full Phase 1 writeup that carries every step at length: the equations of motion derived line by line, why each parameter was chosen and checked against Gillespie, the complete ISO 8608

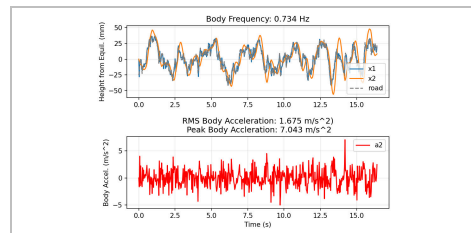
and variance derivations, and all eight benchmark cases with their plots. If you want to see how I actually work through a problem, read that.



RESONANCE ROAD / PASSIVE (TOP) VS SKYHOOK (BOTTOM)



CLASS-C ROAD / PASSIVE (TOP) VS SKYHOOK (BOTTOM)



CLASS-C ROAD / PASSIVE / RMS 1.675 M/S²

Full write-up and photos: benjamintranpugh.com/projects/active-suspension/

Robot Arm 2.0

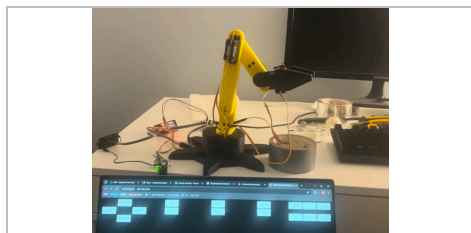
DOF:4 / IK:FULL / MCU:ESP32

SolidWorks, FEA, C++, Python, MATLAB, ESP32

NOV-DEC 2025

A 4-DOF robot arm with full inverse kinematics, browser-based control, and positional memory.

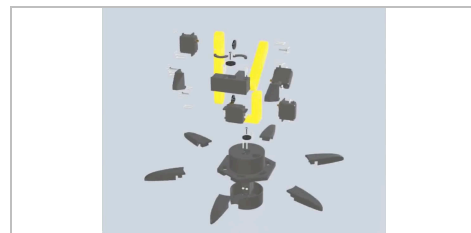
- **Goal:** The goal: a functioning 4-DOF robot arm with full inverse kinematics, smooth motion, easy controls, and working positional memory, a major step up from my earlier 3-DOF build.
- **Method:** I started by researching the math behind inverse kinematics and robotic motion, then modeled the full arm in SolidWorks around accurate models of off-the-shelf components.
- **Outcome:** A working arm with full inverse kinematics, smooth controlled motion, and positional memory, controllable from any device on the network.



THE BUILT ARM



SOLIDWORKS ASSEMBLY



CAD ORBIT / SOLIDWORKS

Full write-up and photos: benjamintranpugh.com/projects/robot-arm-2/

Double Pendulum

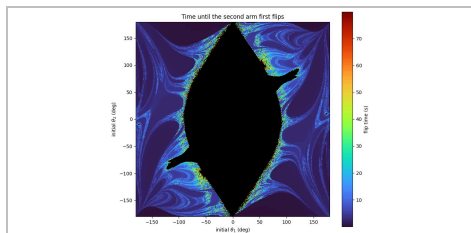
EOM:LAGRANGE / SOLVER:NUMERIC

Python, Lagrangian Mechanics, ODEs, LaTeX, Linear Algebra

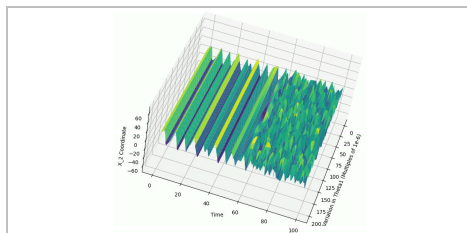
JUL 2026

Deriving the equations of motion from the Euler–Lagrange equation and simulating chaos in Python.

- **Goal:** The double pendulum is one of the simplest systems that produces chaos. I derived its equations of motion from the Euler–Lagrange equation and solved the resulting non-linear ODEs numerically in Python to explore how chaotic it really is.
- **Method:** Derived the equations of motion with Lagrangian mechanics (full derivation typeset in LaTeX); Solved the coupled non-linear ODEs numerically in Python; Swept initial conditions to map flip times and divergence behavior
- **Outcome:** To put into perspective how small $1e-9$ rad is, if you shined two lasers at the moon separated by $1e-9$ rad, the distance between them would be just over a foot.



TIME-TO-FLIP ACROSS INITIAL CONDITIONS / FRACTAL STRUCTURE



200 RUNS, $\Delta\theta_0 = 1e-9$ RAD / DIVERGENCE OVER TIME



EULER–LAGRANGE DERIVATION, TYPESET IN LATEX

Full write-up and photos: benjamintranpugh.com/projects/double-pendulum/

Air Cannon

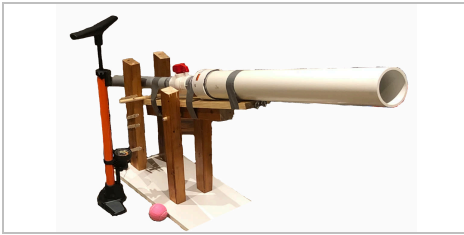
TYPE:PNEUMATIC / SPEC:10M / ACTUAL:60M+

Fusion 360, Onshape, Pneumatics, Fabrication

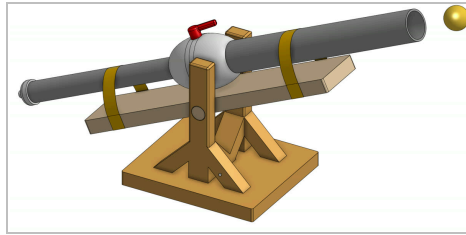
DEC 2023

A pneumatic launcher on needed for a 10 m target. It hit 60+ m.

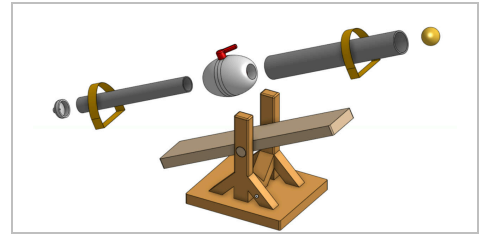
- **Goal:** The assignment was a tennis-ball launcher with a maximum range of 10 meters, any more was supposed to be difficult. We decided that wasn't enough and built a pneumatic cannon that launches projectiles over 60 meters.
- **Method:** The cannon is built from PVC piping, wood, duct tape, and epoxy, pressurized with a bicycle tire pump. Getting there meant pressure research to size the chamber, CAD models in Fusion 360 and Onshape, and a lot of hands-on work, such as hand sawing, PVC solvent welding, and epoxy joints.
- **Outcome:** 60+ meters of range from a hand-pumped chamber, and a hard lesson in the difference between a CAD model and a thing you can actually manufacture.



THE FINISHED CANNON / PRESSURE CHAMBER, VALVE, AND BARREL



CAD MODEL



EXPLODED VIEW

Full write-up and photos: benjamintranpugh.com/projects/air-cannon/

V6 Engine Model

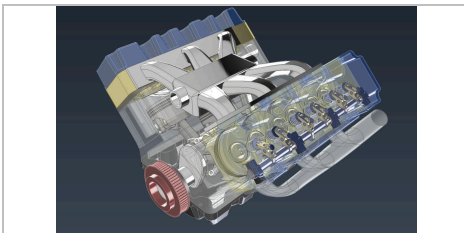
CONFIG:V6 / RENDER:INVENTOR STUDIO

Autodesk Inventor, Rendering, Animation, Large Assemblies

JUL 2025

A detailed V6 engine assembly modeled from real engineering drawings in Autodesk Inventor.

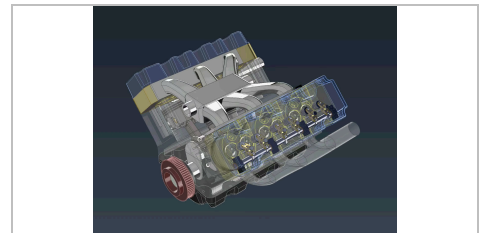
- **Goal:** While researching how different internal combustion engines work, I found engineering drawings of V6 components, so I modeled the whole engine. I had just gotten access to Autodesk Inventor, and this became my crash course in it.
- **Method:** Working from the component drawings, I modeled each part and built up the full assembly, then used Inventor Studio for rendering and animation of the moving engine.
- **Outcome:** A complete, animated V6 model, and a much deeper understanding of what every component in an engine actually does. The orbit render below was animated in Inventor Studio.



FULL ASSEMBLY / VALVETRAIN AND CRANK VISIBLE THROUGH TRANSPARENT HEAD



EXPLODED VIEW



ORBIT RENDER / ANIMATED IN INVENTOR STUDIO

Full write-up and photos: benjamintranpugh.com/projects/v6-engine/